REPORT

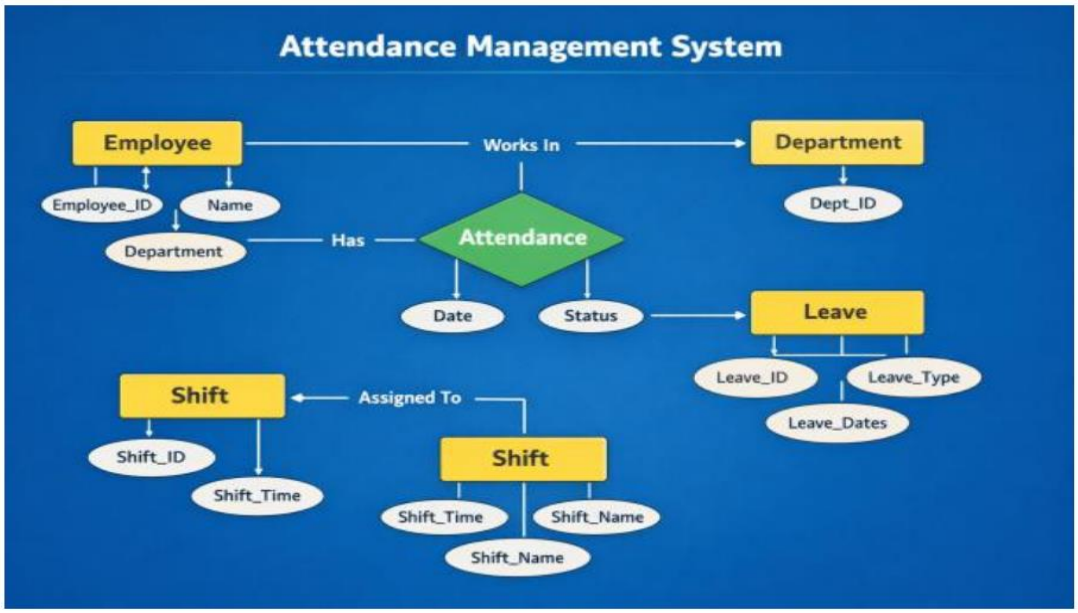
Attendance Management System

Ibrar Ahmed Khan (64292)

1. Core Schema Objects

This section describes the foundational elements of the AttendanceDB database.

1.1 Entity Relationship Diagram



1.2 Tables and Fields

Department

• Department_ID (INT, Primary Key, AUTO_INCREMENT)

• Department_Name (VARCHAR(100), NOT NULL, UNIQUE)

• Location (VARCHAR(100))

Purpose: Stores information about company departments and their physical locations.

Shift

• Shift_ID (INT, Primary Key, AUTO_INCREMENT)
• Shift_Name (VARCHAR(50), NOT NULL)
• Start_Time (TIME, NOT NULL)
• End_Time (TIME, NOT NULL)

Purpose: Stores work shift schedules with their start and end times.

Employee

• Employee_ID (INT, Primary Key, AUTO_INCREMENT)
• Name (VARCHAR(100), NOT NULL)
• Email (VARCHAR(100), UNIQUE)
• Phone (VARCHAR(20))
• Department_ID (INT, Foreign Key referencing Department)
• Shift_ID (INT, Foreign Key referencing Shift)

Purpose: Stores employee personal details, their department assignment, and their assigned work shift.

Attendance

• Attendance_ID (INT, Primary Key, AUTO_INCREMENT)
• Employee_ID (INT, NOT NULL, Foreign Key referencing Employee)
• Attendance_Date (DATE, NOT NULL)
• Status (ENUM: 'Present', 'Absent', 'Leave', NOT NULL)

Purpose: Records each employee's daily attendance status. A composite UNIQUE constraint on (Employee_ID, Attendance_Date) prevents duplicate entries for the same employee on the same day.

Leave_Record

• Leave_ID (INT, Primary Key, AUTO_INCREMENT)
• Employee_ID (INT, NOT NULL, Foreign Key referencing Employee)
• Leave_Type (VARCHAR(50), NOT NULL)
• Start_Date (DATE, NOT NULL)
• End_Date (DATE, NOT NULL)
• Reason (VARCHAR(255))
• Status (ENUM: 'Approved', 'Pending', 'Rejected', DEFAULT 'Pending')

Purpose: Tracks employee leave applications, their types, date ranges, and approval status. A CHECK constraint ensures End_Date is not earlier than Start_Date.

1.3 Keys and Constraints

Primary Keys: Every table has a primary key (e.g., Department_ID, Shift_ID, Employee_ID, Attendance_ID, Leave_ID) with the AUTO_INCREMENT attribute to ensure each record is unique.

Foreign Keys:

• Employee references both Department and Shift via Department_ID and Shift_ID. ON DELETE SET NULL ensures that if a department or shift is removed, the employee's link is cleared without deleting the employee record.

• Attendance references Employee via Employee_ID. ON DELETE CASCADE ensures that if an employee is deleted, all attendance records are automatically removed.

• Leave_Record references Employee via Employee_ID. ON DELETE CASCADE ensures that if an employee is deleted, all leave records are automatically removed.

Check Constraints:

• Leave_Record: A CHECK constraint ensures End_Date >= Start_Date, preventing logically invalid leave date ranges.

• Attendance.Status: An ENUM constraint restricts status values to 'Present', 'Absent', or 'Leave', ensuring data integrity.

Unique Constraints:

• Department.Department_Name: Ensures no two departments share the same name.

• Employee.Email: Ensures all employee email addresses are unique.

• Attendance (Employee_ID, Attendance_Date): A composite unique key prevents duplicate attendance entries for the same employee on the same date.

1.4 Normalization (Up to 3NF)

The AttendanceDB database design has been validated to be in Third Normal Form (3NF):

• 1NF: All tables have atomic, single-valued columns with no repeating groups. Each row is uniquely identified by its primary key.

• 2NF: There are no partial dependencies. Every non-key attribute in each table is fully dependent on its table's entire primary key.

• 3NF: No transitive dependencies exist. For example, the Employee table stores Department_ID and Shift_ID as foreign keys but does not store Department_Name or Shift_Name directly.

First Normal Form (1NF)

All tables have atomic values (no repeating groups). In the AttendanceDB, a composite UNIQUE constraint on (Employee_ID, Attendance_Date) in the Attendance table enforces row uniqueness for daily records.

Second Normal Form (2NF)

Rule: Meet all 1NF requirements AND remove partial dependencies. Non-key attributes must depend on the entire primary key.

Identified Partial Dependencies:

• Employee_Name, Email, Phone depend only on Employee_ID

• Attendance_Date and Status depend fully on Attendance_ID

• Leave_Type, Start_Date, End_Date, Reason, and Status depend only on Leave_ID

Decomposition into 2NF Tables:

Table A: Employee Table

Primary Key: Employee_ID

ID	Name	Email	Phone	Dept_ID	Shift_ID
1	Ahmed Khan	ahmed@company.com	0300-1111111	2	1
2	Sara Ali	sara@company.com	0300-2222222	1	4
3	Usman Tariq	usman@company.com	0300-3333333	2	2
4	Ayesha Noor	ayesha@company.com	0300-4444444	3	4
5	Bilal Hassan	bilal@company.com	0300-5555555	4	1
6	Fatima Zahra	fatima@company.com	0300-6666666	1	4

Table B: Attendance Table

Primary Key: Attendance_ID

Att_ID	Employee_ID	Att_Date	Status
1	1	2026-05-04	Present
2	1	2026-05-05	Present
3	2	2026-05-04	Present
4	2	2026-05-05	Absent
5	3	2026-05-04	Present
6	3	2026-05-05	Leave
7	4	2026-05-04	Present
8	4	2026-05-05	Present
9	5	2026-05-04	Present
10	5	2026-05-05	Present
11	6	2026-05-04	Absent
12	6	2026-05-05	Present

Table C: Leave_Record Table

Primary Key: Leave_ID

Leave_ID	Employee_ID	Leave_Type	Start_Date	End_Date	Reason	Status
1	3	Sick Leave	2026-05-05	2026-05-05	Fever	Approved
2	6	Casual Leave	2026-05-04	2026-05-04	Personal	Approved

Leave_ID	Employee_ID	Leave_Type	Start_Date	End_Date	Reason	Status
3	2	Annual Leave	2026-05-10	2026-05-15	Vacation	Pending

Third Normal Form (3NF)

Meet all 2NF requirements AND remove transitive dependencies. No non-key attribute should depend on another non-key attribute.

Identified Transitive Dependencies:

• In the Employee Table: no transitive dependency exists; Department_Name and Location are stored in the separate Department table, not in Employee.

• In the Leave_Record Table: no transitive dependency exists; employee details such as Name are stored in the Employee table, not directly in Leave_Record.

Decomposition into 3NF Tables (Final Schema):

Department

Primary Key: Department_ID

Dept_ID	Department Name	Location
1	Human Resources	Floor 1, Block A
2	Information Technology	Floor 2, Block B
3	Finance	Floor 3, Block A
4	Marketing	Floor 2, Block C

Shift

Primary Key: Shift_ID

Shift_ID	Shift_Name	Start_Time	End_Time
1	Morning	08:00:00	16:00:00
2	Evening	16:00:00	00:00:00
3	Night	00:00:00	08:00:00
4	General	09:00:00	17:00:00

Employee

Primary Key: Employee_ID | Foreign Keys: Department_ID → Department, Shift_ID → Shift

Emp_ID	Name	Email	Phone	Dept_ID	Shift_ID
1	Ahmed Khan	ahmed@company.com	0300-1111111	2	1
2	Sara Ali	sara@company.com	0300-2222222	1	4

Emp_ID	Name	Email	Phone	Dept_ID	Shift_ID
3	Usman Tariq	usman@company.com	0300-3333333	2	2
4	Ayesha Noor	ayesha@company.com	0300-4444444	3	4
5	Bilal Hassan	bilal@company.com	0300-5555555	4	1
6	Fatima Zahra	fatima@company.com	0300-6666666	1	4

Attendance

Primary Key: Attendance_ID | Foreign Key: Employee_ID → Employee (ON DELETE CASCADE)

Att_ID	Employee_ID	Att_Date	Status
1	1	2026-05-04	Present
2	1	2026-05-05	Present
3	2	2026-05-04	Present
4	2	2026-05-05	Absent
5	3	2026-05-04	Present
6	3	2026-05-05	Leave
7	4	2026-05-04	Present
8	4	2026-05-05	Present
9	5	2026-05-04	Present
10	5	2026-05-05	Present
11	6	2026-05-04	Absent
12	6	2026-05-05	Present

Leave_Record

Primary Key: Leave_ID | Foreign Key: Employee_ID → Employee (ON DELETE CASCADE)

Leave_ID	Emp_ID	Leave_Type	Start_Date	End_Date	Reason	Status
1	3	Sick Leave	2026-05-05	2026-05-05	Fever	Approved
2	6	Casual Leave	2026-05-04	2026-05-04	Personal	Approved
3	2	Annual Leave	2026-05-10	2026-05-15	Vacation	Pending

2. Queries and Implementation

This section details the SQL queries used to retrieve and manipulate data, as implemented in the provided script.

2.1 Joins

Query 1: Employee Attendance with Department

```
SELECT e.Name, d.Department_Name, a.Attendance_Date, a.Status
FROM Attendance a
JOIN Employee e ON a.Employee_ID = e.Employee_ID
JOIN Department d ON e.Department_ID = d.Department_ID;
```

Query 2: Daily Attendance Report

```
SELECT Status, COUNT(*) AS Count
FROM Attendance
WHERE Attendance_Date = '2026-05-05'
GROUP BY Status;
```

Query 3: Monthly Attendance Summary per Employee

```
SELECT e.Name,
    SUM(CASE WHEN a.Status = 'Present' THEN 1 ELSE 0 END) AS Present,
    SUM(CASE WHEN a.Status = 'Absent' THEN 1 ELSE 0 END) AS Absent,
    SUM(CASE WHEN a.Status = 'Leave' THEN 1 ELSE 0 END) AS Leaves
FROM Employee e
LEFT JOIN Attendance a ON e.Employee_ID = a.Employee_ID
GROUP BY e.Employee_ID, e.Name;
```

2.2 Aggregation Functions

Query 4: Employee with Most Absences

```
SELECT e.Name, COUNT(*) AS Absences
FROM Employee e
JOIN Attendance a ON e.Employee_ID = a.Employee_ID
WHERE a.Status = 'Absent'
GROUP BY e.Employee_ID, e.Name
ORDER BY Absences DESC
LIMIT 1;
```

Query 5: Leave Report with Employee Details

```
SELECT e.Name, lr.Leave_Type, lr.Start_Date, lr.End_Date, lr.Status
FROM Leave_Record lr
JOIN Employee e ON lr.Employee_ID = e.Employee_ID;
```

Query 6: Employee Details with Shift and Department

```
SELECT e.Name, d.Department_Name, s.Shift_Name
FROM Employee e
JOIN Department d ON e.Department_ID = d.Department_ID
JOIN Shift s ON e.Shift_ID = s.Shift_ID;
```

2.3 Subqueries

Subqueries allow filtering data based on results from another query. A typical use case in the AttendanceDB schema is to find employees who have been absent, using a subquery to retrieve their attendance records.

Example: Find employees who were absent on 2026-05-05

```
SELECT Name
FROM Employee
WHERE Employee_ID IN (
    SELECT Employee_ID
    FROM Attendance
    WHERE Status = 'Absent'
    AND Attendance_Date = '2026-05-05'
);
```

2.4 Views

View Name: EmployeeAttendanceSummaryView

Purpose: To provide a simplified, reusable query that shows each employee's attendance record along with their name and department, without needing to rewrite the join each time.

```
CREATE VIEW EmployeeAttendanceSummaryView AS
SELECT
    e.Employee_ID,
    e.Name,
    d.Department_Name,
    a.Attendance_Date,
    a.Status
FROM Attendance a
JOIN Employee e ON a.Employee_ID = e.Employee_ID
JOIN Department d ON e.Department_ID = d.Department_ID;

-- Query the view
SELECT * FROM EmployeeAttendanceSummaryView;
```

GitHub Repository

Repository: https://github.com/64292-alt/Attendance-Management-System-